

DOCUMENTO TÉCNICO DE ARQUITECTURA CLOUD: BARBERTECH

1. Nombre del Proyecto y Objetivo

* **Nombre del Proyecto:** BarberTech (Sistema Web Desplegado en la Nube)

* **Objetivo:** Implementar una infraestructura en la nube robusta, segura y escalable para desplegar una aplicación web automatizada, separando de manera independiente las capas de cómputo, almacenamiento de archivos multimedia y la persistencia de datos relacionales.

2. Arquitectura Propuesta y Flujo de Comunicación

El flujo estructurado de comunicación e interconexión de componentes sigue el siguiente patrón lineal y jerárquico:

1. **Usuario:** Se conecta mediante protocolo seguro HTTPS a la Aplicación Web en EC2.
2. **Aplicación Web en EC2:** Consume la lógica de negocio expuesta por el Backend API REST.
3. **Backend API REST:** Realiza consultas y transacciones atómicas seguras hacia AWS RDS (MySQL).
4. **Backend API REST:** Sube y descarga archivos multimedia directamente desde AWS S3.
5. **GitHub Repository:** Sincroniza y despliega el código fuente directamente en la Instancia AWS EC2.

3. Componentes de la Infraestructura Cloud

A. Aplicación Web Funcional (Frontend)

- * Módulo de registro e inicio de sesión seguro de usuarios mediante tokens web eficientes (JWT).
- * Vistas dinámicas optimizadas para la navegación de catálogos y consulta de inventarios en tiempo real (mínimo 30 productos mapeados dinámicamente).
- * Control de accesos y visualización basada en roles (RBAC) con precios diferenciados para Consumidor Final (B2C) y Barbero Profesional/Mayorista (B2B).
- * Formulario público integrado para la captación directa de datos de clientes e interesados (Leads).

B. Backend API REST (Endpoints Implementados)

- * **POST /api/auth/register:** Registro de nuevos usuarios y hashing criptográfico de contraseñas.
- * **POST /api/auth/login:** Autenticación de usuarios y generación de tokens JWT de sesión.
- * **GET /api/products:** Consulta dinámica del catálogo unificado con lógica transaccional de precios según rol.

- * **`POST /api/orders`**: Procesamiento e inserción transaccional de pedidos de forma segura.
- * **`POST /api/leads`**: Almacenamiento e inserción de datos capturados en formularios públicos.
- * **`POST /api/files/upload`**: Carga directa de archivos binarios e imágenes hacia el bucket de AWS S3.

C. Base de Datos en AWS RDS (Estructura Relacional ACID)

Se utiliza el servicio administrado AWS RDS (MySQL) alojado en subredes privadas, empleando obligatoriamente el motor de almacenamiento InnoDB para garantizar la atomicidad, consistencia, aislamiento y durabilidad (ACID). El esquema cuenta con 5 tablas base estrechamente relacionadas mediante claves foráneas restringidas:

- * **`users`**: Control de credenciales (password_hash) y roles de usuario.
- * **`products`**: Entidad maestra de inventarios base.
- * **`prices`**: Desacoplamiento de precios según el tipo de rol (`CLIENTE` / `MAYORISTA`).
- * **`orders`**: Cabecera transaccional para el procesamiento de compras de los usuarios.
- * **`leads`**: Registro indexado de prospección comercial.

D. Almacenamiento Descentralizado en AWS S3

Uso exclusivo del servicio independiente AWS S3 (Simple Storage Service) para hospedar imágenes de catálogo de forma desacoplada, manteniendo la instancia EC2 ligera. Las imágenes físicas residen en el bucket y la base de datos únicamente almacena la cadena de texto con la URL pública de acceso directo.